

SALUS SECURITY

AUG 2026



CODE SECURITY ASSESSMENT

ALLOCO ETP

Overview

Project Summary

- Name: Alloco-ETP
- Platform: EVM-compatible chains
- Language: Solidity
- Audit Range: See [Appendix - 1](#)

Project Dashboard

Application Summary

Name	Alloco-ETP
Version	v2
Type	Solidity
Dates	Aug 20 2026
Logs	Aug 20 2026, Aug 20 2026

Vulnerability Summary

Total High-Severity issues	0
Total Medium-Severity issues	1
Total Low-Severity issues	2
Total informational issues	1
Total	4

Contact

E-mail: support@salusec.io

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

Content

Introduction	4
1.1 About SALUS	4
1.2 Audit Breakdown	4
1.3 Disclaimer	4
Findings	5
2.1 Summary of Findings	5
2.2 Notable Findings	6
1. Centralization risk	6
2. Claim Signatures Never Expire	7
3. Nominal Amounts Misrepresent Value Limits	8
2.3 Informational Findings	9
4. Initial Signer and Treasury Configuration Is Not Evented	9
Appendix	10
Appendix 1 - Files in Scope	10

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (<https://t.me/salusec>), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Centralization risk	Medium	Centralization	Acknowledged
2	Claim Signatures Never Expire	Low	Data Validation	Acknowledged
3	Nominal Amounts Misrepresent Value Limits	Low	Business Logic	Acknowledged
4	Initial Signer and Treasury Configuration Is Not Evented	Informational	Logging	Acknowledged

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Centralization risk	
Severity: Medium	Category: Centralization
Target: - src/ETPSubscription.sol	

Description

The owner of the ETPSubscription contract can set the treasury address, which receives subscription payments and supplies funds for claims and refunds.

If the owner's private key is compromised, an attacker can replace the treasury with an attacker-controlled address. Future subscription payments would be redirected, and outstanding claims or refunds could fail because the replacement treasury may not hold the required tokens or allowances.

src/ETPSubscription.sol:L798-L805

```
function setTreasury(address newTreasury) external onlyOwner notBlacklisted {
    require(newTreasury != address(0), ZeroTreasury());

    address oldTreasury = treasury;
    treasury = newTreasury;

    emit TreasuryUpdated(oldTreasury, newTreasury);
}
```

Recommendation

We recommend transferring ownership to a multi-signature account protected by a timelock. This ensures that no single party can unilaterally redirect treasury flows and gives users advance notice of treasury changes.

Status

This issue has been acknowledged by the team.

2. Claim Signatures Never Expire

Severity: Low

Category: Data Validation

Target:

- src/ETPSubscription.sol

Description

Claim signatures bind the recipient, request nonce, assets, amounts, contract, and chain, but the signed payload contains no deadline.

Nonce deletion prevents replay after a successful claim, but it does not expire an unused authorization. Revoking a stale single request requires an alternative signed flow or rotation of the global signer.

src/ETPSubscription.sol:L665-L684

```
function claim(  
    address _token,  
    address _etpToken,  
    uint256 nonce,  
    uint256 tokenAmount,  
    uint256 principalAmount,  
    bytes calldata signature  
) external nonReentrant whenNotPaused notBlacklisted onlySupportedToken(_token)  
    onlySupportedEtpToken(_etpToken) {  
    bytes32 digest = _claimDigest(msg.sender, _token, _etpToken, nonce, tokenAmount,  
    principalAmount);  
  
    require(_isValidSignature(digest, signature), InvalidSignature());  
}
```

Recommendation

Include an expiry in every claim authorization, bind it into the signed digest, and reject signatures after the deadline. Provide a documented per-request cancellation or replacement path when quotes must be revoked early.

Status

This issue has been acknowledged by the team.

3. Nominal Amounts Misrepresent Value Limits

Severity: Low

Category: Business Logic

Target:

- src/ETPSubscription.sol

Description

The subscription path normalizes payment tokens only by decimals and then uses the normalized amount to consume shared quota.

The shared whitelist and public-period limits therefore do not measure the economic value accepted by the protocol.

If a supported stablecoin trades above par, a user can consume a USD-denominated quota with fewer counted dollars than the actual value deposited. A depeg below par has the opposite effect and unnecessarily consumes capacity.

src/ETPSubscription.sol:L495-L510

```
function subscribe(address _token, address _etpToken, uint256 amount)
    external
    nonReentrant
    whenNotPaused
    notBlacklisted
    onlySupportedToken(_token)
    onlySupportedEtpToken(_etpToken)
    returns (uint256 nonce)
{
    // amount must be larger than 0.
    require(amount > 0, ZeroAmount());
    // normalized token amount.
    uint256 normalizedAmount = _normalizeTokenAmount(_token, amount);
    // consume the quota.
    _consumeSubscriptionQuota(msg.sender, normalizedAmount);
}
```

Recommendation

Price shared value limits with a robust oracle or apply conservative per-token haircuts. Alternatively, maintain separate token-denominated quotas and pause a token when it materially depegs.

Status

This issue has been acknowledged by the team.

2.3 Informational Findings

4. Initial Signer and Treasury Configuration Is Not Evented

Severity: Informational

Category: Logging

Target:

- src/ETPSubscription.sol

Description

The constructor assigns the initial signer and treasury but emits no event for either value. The values remain readable from contract storage and deployment calldata; however, the absence of a dedicated initialization event makes deployment-time configuration less convenient for indexers and monitoring systems.

src/ETPSubscription.sol:L459-L469

```
constructor(address signer_, address treasury_, string memory initialEtpName_, string
memory initialEtpSymbol_)
    Ownable(msg.sender)
{
    require(signer_ != address(0), ZeroSigner());
    require(treasury_ != address(0), ZeroTreasury());

    signer = signer_;
    treasury = treasury_;
    // create the first etp token.
    _createEtpToken(initialEtpName_, initialEtpSymbol_);
}
```

Recommendation

Emit an initialization event containing the initial signer and treasury addresses, or document an equivalent deployment-indexing procedure.

Status

This issue has been acknowledged by the team.

Appendix

Appendix 1 - Files in Scope

This audit covered the following files:

File	SHA-1 hash
src/ETPSubscription.sol	2d7ebc8834bca71c01a423a94e701687bdf0855d
src/ETPToken.sol	4ce2eaa52cf161a71544f33f0bbba10192b89e2b